

Examen-3.pdf



qilnx



Estructuras de Datos Avanzadas



3º Doble Grado en Ingeniería Informática e Ingeniería del Software



**Escuela Técnica Superior de Ingeniería Informática.
Campus de Móstoles
Universidad Rey Juan Carlos**



[Accede al documento original](#)

 **Gemini**

**Dale al play y
estudia a tu ritmo**

Convierte estos apuntes en un track pegadizo
que me ayude a memorizar las palabras clave

Apuntes Tema 3

 PDF



Razonamiento ▾



Pruébalo



Estructuras de Datos Avanzadas

Examen Parcial

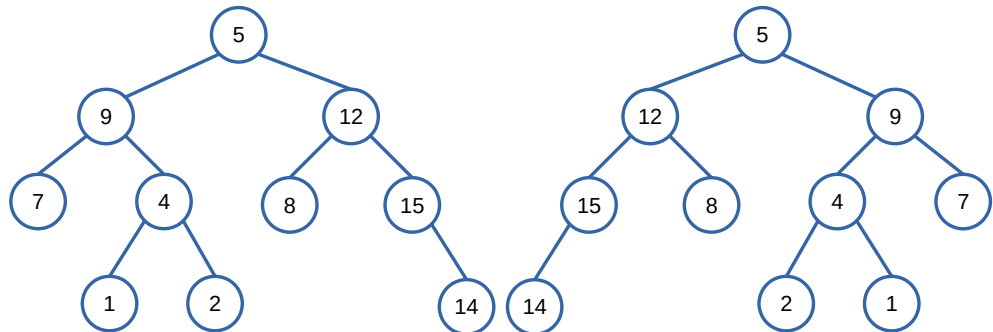
5 de febrero de 2021

- La duración del examen es de 120 minutos.
- Desde el campus virtual hay que descargar un archivo ZIP que contiene el esqueleto de los dos ejercicios del examen así como los test unitarios correspondientes. Se recomienda crear un proyecto de Netbeans vacío; copiar dentro de la carpeta del proyecto la carpeta src y la carpeta test proporcionadas; añadir las librerías JUnit y Hamcrest; y finalmente cerrar y volver a abrir Netbeans.
- El alumno debe generar un fichero ZIP con el directorio en el que esté el código del alumno y subirá el fichero ZIP a la actividad examen práctico parcial del aula virtual asegurándose de que entrega el código desarrollado durante la prueba.

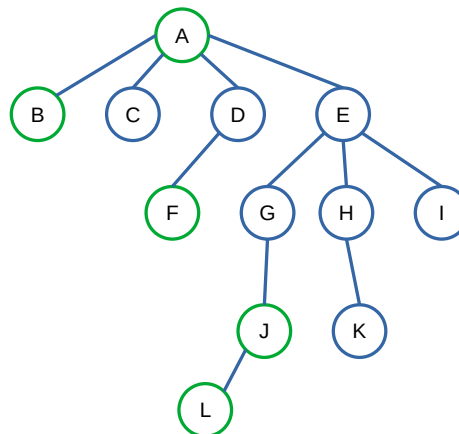
Ejercicio 1 [3 puntos]

Se desea ampliar la funcionalidad de los árboles, para ello se ha creado una nueva clase `MoreFunctionality`, en el paquete `material.tree`, que posee dos nuevos métodos:

- el método `checkMirror` que dados dos árboles binarios determina si son uno el reflejo del otro (1,5 puntos). Los árboles de la figura son uno el reflejo del otro:



- el método `leftView` que dado un árbol n-ario nos devuelve los elementos que se verían desde su lado izquierdo. (1,5 puntos).



En el árbol de la figura la secuencia de los nodos que debe devolver el método serían: A, B, F, J, L. Que son los nodos del árbol que se verían desde la izquierda.

NOTA: Las declaraciones de dichos métodos están en el fichero `MoreFunctionality.java`. Sus test están en el fichero `MoreFunctionality Test.java`. Se prohíbe modificar de cualquier forma las declaraciones de los métodos suministrados en el proyecto. Está permitido añadir métodos privados o clases adicionales si se considera necesario.

Ejercicio 2 [4 puntos]

En el ISP URJC Network Corporation S.A. se utiliza internamente el algoritmo RIP para actualizar las rutas de su red. Dicho algoritmo se resume en las siguiente líneas:

- Cada router puede estar conectado a otros routers dando lugar a una *red*.
- Los *routers* que están conectados entre sí se intercambian *mensajes* con información acerca de los destinos alcanzables a través de ellos.
- Cada mensaje M incluye el vecino que informa V, un destino alcanzable D y un coste asociado A.
- Cuando un *router* recibe un mensaje de su vecino M(V,D,A), debe comprobar si el destino anunciado D es conocido o no por él.
 - Si no lo conoce lo guarda como un nuevo destino accesible desde el vecino V que le informó y le apunta un coste de uno más el coste del vecino (1+A).
 - Si el destino anunciado es conocido, debe comprobar si la nueva ruta tiene menor coste que la que ya tiene. Si es así, modificará los datos del vecino a V y el coste que tenga asociado a ese destino (A+1).

Se pide:

a) Implemente en las clases `Network` y `Router` las estructuras de datos necesarias para la resolución de este ejercicio. Se valorará especialmente la eficiencia de la representación elegida.

[0,5 puntos]

b) Implemente los siguientes métodos de la clase `Router`:

- `advertise` que devuelve una lista de mensajes con todos los destinos alcanzables desde el *router*.
- `receiveMessage` que recibe una lista de mensajes de otro *router* y aplica el algoritmo RIP descrito arriba. Este método devuelve TRUE si se produce alguna actualización de la información del router o FALSE en caso contrario.
- `KnownDestination` que recibe un destino y devuelve TRUE si forma parte de sus destinos conocidos y FALSE en caso contrario.

[1,5 puntos]

c) Implemente el método `connectNewRouter` de la clase `Network`. Dicho método recibe un nuevo *router* R y una lista de *routers* vecinos L, a los que estará conectado directamente. Este método llama al método `receiveMessage` de los routers de L enviándole la información generada por el método `advertise` de R. Este proceso se repite de forma recursiva por cada router R' de L que devuelva TRUE al invocar `receiveMessage`.

[2 puntos]

NOTA: En el paquete `URJCNetwork` se proporciona la clase `Message` completamente implementada y la declaración de los métodos de las clases `Router` y `Network`. Sus test están en el fichero `NetworkTest.java`. Se prohíbe modificar de cualquier forma las declaraciones de los métodos suministrados en el proyecto. Está permitido añadir métodos privados adicionales si se considera necesario.